

Musterlösungen

01 | a) `for (int i = 0; i < 10; i++) { }`

Dies ist die korrekte Syntax für eine einfache `for`-Schleife, die bei `i = 0` beginnt und fortfährt, solange `i < 10`, wobei `i` nach jedem Durchlauf um 1 erhöht wird.

02 | c) Die Schleife wird nie ausgeführt.

Wenn die Bedingung nie erfüllt ist (z.B. `i < 0` bei einer Schleife, die von 0 bis 10 zählt), wird der Code im Schleifen-Body nie ausgeführt.

03 | c) Beide oben genannten Methoden sind korrekt.

Beide Methoden sind gültig, um durch ein Array zu iterieren: Die erste Methode verwendet das erweiterte `for`, die zweite Methode verwendet den klassischen `for`-Index.

04 | a) Er erhöht den Wert von `i` um 1.

Der Ausdruck `i++` erhöht den Wert von `i` um 1, nachdem der aktuelle Schleifendurchlauf abgeschlossen ist.

05 | a) `for (int i = 10; i >= 0; i--) { }`

Eine Schleife, die rückwärts zählt, verwendet `i--`, um `i` bei jedem Durchlauf um 1 zu verringern.

06 | a) `for (int i = 1; i <= 5; i++) { System.out.println(i); }`

Diese Schleife gibt die Zahlen von 1 bis 5 aus, weil sie bei `i = 1` beginnt und bis `i = 5` läuft.

07 | a) Der Schleifen-Index wird weiterhin automatisch verändert.

Wenn der Schleifen-Index innerhalb der Schleife manuell geändert wird, verändert er sich nur lokal, aber die automatische Inkrementierung (z.B. `i++`) bleibt bestehen.

08 | a) `for (int i = 0; i < array.length; i++) { for (int j = 0; j < array[i].length; j++) { } }`

Diese Syntax ermöglicht die Iteration durch ein zweidimensionales Array, indem zuerst durch die äußeren und dann durch die inneren Arrays iteriert wird.

09 | a) `break`;

Die `break`-Anweisung beendet die Schleife vorzeitig, wenn eine bestimmte Bedingung erfüllt ist.

10 | a) Sie überspringt den aktuellen Durchlauf der Schleife und fährt mit dem nächsten fort.

Die `continue`-Anweisung überspringt den restlichen Code im aktuellen Schleifendurchlauf und fährt mit der nächsten Iteration fort.
